TOBB UNIVERSITY OF ECONOMICS AND TECHNOLOGY DEPARTMENT OF COMPUTER ENGINEERING

BİL 481 - SOFTWARE ENGINEERING PROJECT ASSIGNMENT 2: QUALITY ASSURANCE (QA) PLAN

PROJECT NAME: SWAPLY

AUTHORS:

Arda Burak Zararsız

Yusuf Çakır

Yusuf Enes Kütük

Eren Öz

Furkan Yılmaz

DATE: 05.03.2026

Table of Contents Document-Specific

1. Task Matrix 2
2. Quality Assurance Strategy 3
3. Quality Factors & Metrics 3
4. Test Plan 4

Quality Assurance Strategy & Methodologies: Arda Burak Zararsız

Quality Factors & Metrics Definition: Eren Öz, Arda Burak Zararsız

Test Plan & Test Cases Implementation: Yusuf Çakır, Yusuf Enes Kütük

Defect Management & Bug Tracking: Furkan Yılmaz

Document Review & Formatting: Eren Öz

Quality Assurance Strategy

Overview: The primary objective of this Quality Assurance (QA) plan is to ensure that Swaply operates correctly, securely, and reliably across targeted platforms (Android and iOS) before final deployment. Our approach integrates testing directly into the development lifecycle: testing is conducted continuously as features are built, rather than waiting until the end of the project. We will utilize peer testing, where code is tested by a team member other than the original author, to ensure objective evaluation.

Testing Methodologies: Unit Testing: Testing individual utility functions and backend logic in isolation (e.g., the matching algorithm and distance calculation for location-based search). Component Testing: Verifying that UI components (e.g., Login forms, Profile editing) render correctly and handle user inputs properly. Integration Testing: Testing the data flow between the React Native frontend and the Firebase backend (e.g., ensuring a matched skill successfully triggers a Firestore document update and an FCM notification). Usability Testing: Conducting manual tests with peers to ensure the interface is intuitive and core functions are reachable within the target of 3 clicks.

Automated vs. Manual Testing: Automated Testing: We will automate unit and component tests using Jest and the React Native Testing Library. This will primarily cover utility functions, data validation (e.g., verifying @edu.tr email formats), and basic UI rendering. Manual Testing: End-to-end flows, cross-platform compatibility, and location-based features (Match-Map) will be tested manually. We will use real devices via Expo Go (Android 10+ and iOS 14+ devices) to manually execute integration test cases and verify real-time chat functionality.

Quality Factors & Metrics

Each team must define quality factors to objectively evaluate the success of the software. We have defined four key quality factors aligned with our Non-Functional Requirements:

Performance: System response time for screen transitions and database queries. Measurement Metric: Average response time (ms) - Target is under 3000ms.

Security: Protection of user data and private messages from unauthorized access. Measurement Metric: Number of unauthorized read/write attempts detected via Firebase Emulator.

Usability: Ease of use for university students meeting modern mobile standards. Measurement Metric: User satisfaction score from peer surveys (Target: >70% user-friendly).

Maintainability: Ease of modifying the React Native codebase and Firebase structure. Measurement Metric: Code complexity score (e.g., Cyclomatic Complexity) measured per module using ESLint complexity rules; target average complexity score below 10 per function.

Test Plan

Test Cases (Below are 5 core test cases representing critical functional requirements):

TC-01 / FR1 (Auth): Verify @edu.tr restriction.

Input/Steps:

1) Enter user@gmail.com.

2) Enter valid password.

3) Click Register. Expected Result: Registration is rejected, error message indicates only @edu.tr is allowed. Status: Pending.

TC-02 / FR3 (Match): Verify automatic match notification.

Input/Steps:

1) User A adds "Python" to Teach.

2) User B adds "Python" to Learn. Expected Result: System identifies overlap, triggers FCM notification to both Users A and B. Status: Pending.

TC-03 / FR4 (Chat): Verify real-time messaging.

Input/Steps:

1) User A opens chat with B.

2) Sends "Hi".

3) User B has chat open. Expected Result: Message "Hi" appears on User B's screen instantly without refreshing. Status: Pending.

TC-04 / FR6 (Location): Verify location-based map display.

Input/Steps:

1) Enable GPS.

2) Open Map Screen. Expected Result: Matched users within a 5 km radius are rendered as pins on the map. Status: Pending.

TC-05 / FR8 (Rating): Verify rating submission.

Input/Steps:

1) Complete a swap.

2) Select 5 stars.

3) Enter comment.

4) Submit. Expected Result: The target user's average rating in Firestore updates accurately. Status: Pending.

Bug Tracking and GitHub Integration: Bugs and defects will be strictly managed and tracked using GitHub Issues. This ensures transparency and aligns with our version control strategy.

Reporting: When a bug is found, a GitHub Issue must be created containing: Title, Steps to Reproduce, Expected vs. Actual Behavior, Severity (Critical/High/Medium/Low), and Environment. Resolution Lifecycle: Open -> In Progress -> Fixed -> Verified -> Closed. Commit Message Traceability: Every code change related to testing or fixing bugs MUST be referenced in GitHub commit messages. Clear commit messages should indicate what was added or modified. Developers must use the following standard format to ensure individual contributions are visible:

- Fix: Resolve issue with email validation in Login (Fixes #12)
- Test: Add Jest unit tests for match algorithm calculation
- Chore: Update QA test cases for FR4 chat implementation